

DSU CAST AUTONOMOUS RESEARCH PROGRAM

Week 1 Intern Task Guide | June 15–19, 2026

Python Foundations + Linux Shell Fundamentals

Overview

This is not a class. You are a researcher. Week 1 is about getting your tools sharp so you can do real work with Spot and the drones. By Friday you should be able to write Python scripts, navigate a Linux environment, shell into a remote system, and establish automated connections — the exact same workflow you will use every day with the hardware.

YOUR JOB

Complete every task. Do not wait to be told twice.
Stuck on a technical blocker for more than 20 minutes? Follow the escalation path in Teams.
Accountability begins daily at 10:00 AM.

Communication & Escalation Path

You have a dedicated Microsoft Teams channel. You are surrounded by talented peers, 2 graduate students, and technical leads. When you hit a blocker, use this workflow in order:

- **Self-Heal:** Use the provided video indexes and documentation to troubleshoot first.
- **Peer Support:** Post the exact command or code you ran, the error output, and what you tried in Teams.
- **Escalate:** If still unresolved after 10 minutes of peer collaboration, tag a Grad Student or Tech Lead.

Tool Installation — VS Code & Docker

Complete these installs before Day 1. Do not show up Monday without them.

Visual Studio Code

Download: code.visualstudio.com/download

- Run the installer and accept all defaults.
- Check both boxes: Add to PATH and Register as default editor.
- Verify: open a terminal and run `code --version`

Docker Desktop

Docker is the Linux environment your container will run in all week.

IMPORTANT

Do not skip the verification step. If docker run hello-world does not produce output, Docker is not working and you need to fix it before Monday.

macOS	Windows
Step 1 — Check your chip Apple menu → About This Mac. Note Intel or Apple M1/M2/M3. <i>This determines which installer you download.</i>	Step 1 — Enable WSL 2 (required) Open PowerShell as Administrator and run: <code>wsl --install</code> <i>Restart when prompted. WSL 2 is the Linux engine Docker needs.</i>
Step 2 — Download Docker Desktop Select the correct installer for your chip: <code>Apple Silicon → Docker Desktop for Mac (Apple Silicon)</code> <code>Intel → Docker Desktop for Mac (Intel)</code> <i>docs.docker.com/desktop/install/mac-install/</i>	Step 2 — Download Docker Desktop Go to Docker's download page and click Docker Desktop for Windows: <i>docs.docker.com/desktop/install/windows-install/</i>
Step 3 — Install Open the .dmg file. Drag Docker into Applications. Launch Docker.	Step 3 — Run the installer Run Docker Desktop Installer.exe. Keep 'Use WSL 2 instead of Hyper-V' checked.
Step 4 — Allow permissions Enter your Mac password when prompted. Allow the helper tool install.	Step 4 — Restart Click Close and restart. Docker Desktop launches automatically after reboot.
Step 5 — Wait for startup Watch the menu bar icon. Wait until it shows 'Docker Desktop is running'.	Step 5 — Accept the agreement Accept the service agreement. Wait for the whale icon in your system tray to stop animating.
Step 6 — Verify Open Terminal and run: <code>docker run hello-world</code> <i>You should see: 'Hello from Docker!'</i>	Step 6 — Verify Open PowerShell or Command Prompt and run: <code>docker run hello-world</code> <i>You should see: 'Hello from Docker!'</i>

STUCK?

Post a screenshot of your error in Teams before Monday. Tag a grad student. Do not show up without Docker working.

Before You Start — Container Setup

Run these commands in your terminal (macOS: Terminal, Windows: PowerShell or WSL terminal).

Step 1 — Launch the container with port forwarding & shared volume:

```
docker run -it -p 2222:22 -v ~/cast-work:/workspace ubuntu:22.04 /bin/bash
```

Step 2 — Inside the container, install core dependencies:

```
apt-get update && apt-get install -y python3 python3-pip openssh-server curl
wget
```

Step 3 — Start & configure SSH (required — the capstone is triggered over SSH):

openssh-server is installed but it is NOT running yet, and root login is off by default. Inside the container, run these once to bring it up:

```
mkdir -p /run/sshd
echo 'PermitRootLogin yes' >> /etc/ssh/sshd_config
passwd root # set a root password you will remember
service ssh start # starts the SSH server
```

Then, from your HOST machine (outside Docker), set up password-less access — this is the exact benchmark you are graded on:

```
ssh-keygen -t ed25519 # press Enter through every prompt
ssh-copy-id -p 2222 root@localhost
ssh root@localhost -p 2222 'echo SSH OK'
```

IMPORTANT — keep the container running. If you type 'exit' in the container shell it STOPS and SSH will refuse the connection. Open a second terminal on your host for the SSH trigger, or detach with Ctrl-P then Ctrl-Q instead of exiting.

NOTE

Everything you build this week goes in /workspace. This maps directly to ~/cast-work on your laptop.

Video Reference Index 1 — Linux & Shell (Ranked by Spot Priority)

Source: The 50 Most Popular Linux & Terminal Commands — Colt Steele (freeCodeCamp, 5 hrs). Jump directly to the chapters you need. Do not watch from the beginning.

Full video: youtube.com/watch?v=ZtqBQ68cfJc

Priority	Skill	Timestamp — Start Here	Why It Matters for Spot
Tier 1 — Mission Critical for Spot & Drone Operations			
#1	SSH & remote shell access	su (switch user) — 4:03:47 sudo — 4:08:32	Spot runs Linux onboard. Every command is executed over SSH. Non-negotiable.
#2	Process management — ps, kill, killall, jobs	ps — 2:47:32 kill — 2:54:02 killall — 3:00:13 jobs, bg, fg — 3:01:37	Killing a runaway script on a moving robot is a safety skill. Know this cold.
#3	File permissions — chmod, chown	Understanding permissions — 4:31:08 chmod — 4:47:15 chown — 4:21:54	SDK scripts must be executable. Permissions errors are the #1 silent failure.

#4	Shell scripting — bash, loops, variables	Shell Scripting Crash Course — separate video expansions — 2:06:59	Mission automation on Spot runs through bash. Startup sequences, payload init — all shell scripts.
Tier 2 — High Importance for Daily Research Work			
#5	Log monitoring — tail -f, grep	tail — 1:33:02 grep — 2:32:10	When a mission fails, you read the logs. tail -f on a live log is how you debug without stopping the robot.
#6	Network identification — ip, ping	who / user context — 4:01:49 sudo — 4:08:32	Finding Spot's IP and verifying network connectivity are pre-flight steps before every session.
#7	find & xargs	find — 2:21:01 xargs — 3:42:48	Batch-processing mission log files is a daily data management task on long-running research.
#8	tar, gzip — archiving mission data	gzip / gunzip — 3:09:40 tar — 3:15:27	Mission data packages — logs, point clouds, images — get compressed and transferred off the robot.
Tier 3 & 4 — Foundation Skills (Learn First, Master Fast)			
#9	Filesystem — ls, cd, pwd, mkdir, rm	pwd — 0:39:05 ls — 0:41:07 cd — 0:49:21 mkdir — 1:00:40 rm — 1:13:05	Table stakes. Pick this up in the first 90 minutes — then move on.
#10	Piping, redirection, sort, grep	piping — 1:53:52 redirecting > >> — 1:36:02	Woven into every tier above. Learn it early, use it forever.
#11	nano, alias, man, history	man — 0:32:34 nano — 3:23:36 alias — 3:31:17 history — 2:44:04	Edit configs on the robot over SSH, build shortcuts, read docs. Quality of life that compounds fast.

Video Reference Index 2 — Python (Ranked by Spot SDK Priority)

Source: Python Tutorial for Beginners — freeCodeCamp (4+ hrs). All chapters come from a single video. Jump directly to what you need — the Spot SDK is Python, so this is not optional background knowledge.

Full video: youtube.com/watch?v=rfscVS0vtbw

Priority	Skill	Timestamp — Start Here	Why It Matters for Spot
Tier 1 — Mission Critical: The Spot SDK is Built on These			

#1	Functions — def, parameters, return	Using functions — ~1:35:00 Return statement — ~1:50:00	Every SDK call is a function. Commanding Spot to stand, walk, or capture an image is a function call.
#2	Classes and objects	Classes & objects — ~3:05:00 Class functions — ~3:35:00 Inheritance — ~3:50:00	The entire Spot SDK is object-oriented. Robot, LeaseClient, CommandClient — all class instances. This is non-negotiable.
#3	Error handling — try/except	Catching errors — ~2:45:00	Spot throws exceptions constantly — network drops, lease conflicts, estop triggers. An unhandled exception on a moving robot is a safety incident.
#4	While loops — loop guards, break	While loops — ~2:05:00	Mission loops, sensor polling, waiting for robot state — all while loops. The estop monitor is a while loop with a kill condition.
#5	If statements and comparisons	If statements basics — ~1:55:00 If statements advanced — ~2:00:00	'If battery below 20%, return to dock.' 'If obstacle detected, halt.' Every behavioral decision in Spot is an if statement.
Tier 2 — High Importance for SDK Work			
#6	Dictionaries — key-value pairs	Dictionaries — ~2:30:00	Spot's telemetry, image metadata, and mission configs all parse into dict-like structures. You'll read and write dicts constantly.
#7	File I/O — reading, writing, CSV	Reading files — ~2:55:00 Writing & appending files — ~3:00:00	Mission telemetry logs to CSV. Config files get read from disk. The sensor pipeline from Day 4 is exactly this at production scale.
#8	Modules and imports	Modules — ~3:15:00	The SDK is imported. Every Spot script starts with bosdyn imports. You cannot use any SDK without understanding how imports work.
#9	For loops — iterating collections	For loops — ~2:20:00	Looping over image sources, waypoints, sensor readings, log entries. Any list of robot data you process uses a for loop.
Tier 3 & 4 — Foundation Skills			
#10	Lists — storage, indexing, methods	Lists — ~1:00:00 List functions — ~1:10:00	Waypoint lists, image queues, sensor buffers, command sequences — lists are the container for almost everything sequential in a mission.
#11	Variables and data types	Variables — ~0:12:00 Strings — ~0:20:00	Robot hostname = string. Battery level = float. Motor powered =

		Numbers — ~0:30:00	boolean. The atoms of every script.
#12	Tuples — immutable coordinates	Tuples — ~1:20:00	Spot uses tuples for (x, y, z) positions and quaternions (x, y, z, w) in navigation. You will encounter these in waypoint data.

Daily Tasks

Monday — Day 1

Linux Orientation & Local Environment Setup

 **Daily Standup 10:00 AM** — Program orientation, team introductions, and setting expectations.

Start with Tier 3/4 Linux skills — foundation only. These should take no more than 90 minutes. The rest of the day is hands-on tasks.

► File System Control

- Create a nested directory structure /workspace/week1/day1/ using a single command.
- Move, rename, and delete files cleanly via terminal using pwd, ls, cd, mkdir, rm, cp, mv.

► Permissions Mastery

- Understand file permissions (ls -la).
- Use chmod and chown to restrict a script so only the owner can read/write it, but anyone can execute it.
 - Jump straight to 4:31:08 in the Linux video — don't wait until the end of the week.

► Automation Basics

- Write a shell script (hello.sh) that prints your name and today's date.
- Modify it to output that data into a unique log file within your workspace.

Reference — Linux video timestamps (Tier 3/4 + permissions)

- pwd, ls, cd (0:39–0:49): youtube.com/watch?v=ZtqBQ68cfJc&t=2345
- mkdir, rm (1:00–1:21): youtube.com/watch?v=ZtqBQ68cfJc&t=3640
- Understanding permissions (4:31:08): youtube.com/watch?v=ZtqBQ68cfJc&t=16268
- chmod (4:47:15): youtube.com/watch?v=ZtqBQ68cfJc&t=17235

Tuesday — Day 2

Treating the Container as a Remote Robot (SSH & Networking)

 **Daily Standup 10:00 AM** — What did you accomplish yesterday? What are you focusing on today? What blockers are in your way?

Today covers Tier 1 Linux skills. Before touching actual hardware, configure your Docker container to act as a remote server — the exact same workflow as accessing Spot.

► Network Identification

- Use `ip a` or `ip route` to identify network interfaces. Use `ping` to verify connectivity.

► Process Isolation & Kill-Switches

- Run an infinite loop background script.
- Locate its PID using `ps aux` or `top`. Instantly terminate it using `kill -9`.
 - This is a safety-critical skill for stopping runaway scripts on Spot.

► SSH Server + Remote Connection Test

- Start SSH inside your container: `service ssh start`. Set a password: `passwd`
- From a fresh terminal on your host machine, successfully shell in:

```
ssh root@localhost -p 2222
```

► Passwordless Automation — The Hardware Standard

- Generate an SSH keypair: `ssh-keygen`
- Append your public key into the container's `/root/.ssh/authorized_keys`.
- Prove you can now connect without a password.
 - This is how Spot and drone systems authenticate in production.

Reference — Linux video timestamps (Tier 1)

- **ps (2:47:32)**: youtube.com/watch?v=ZtqBQ68cfJc&t=10052
- **kill (2:54:02)**: youtube.com/watch?v=ZtqBQ68cfJc&t=10442
- **jobs, bg, fg (3:01:37)**: youtube.com/watch?v=ZtqBQ68cfJc&t=10897
- **sudo (4:08:32)**: youtube.com/watch?v=ZtqBQ68cfJc&t=14912

Wednesday — Day 3

Python Foundations — Functions, Loops, Classes

 **Daily Standup 10:00 AM** — Be ready with yesterday's SSH demo. Show, don't tell.

Run Python inside the container. Today covers Python Tiers 1 and 2 — the concepts the Spot SDK is built on. Classes and functions are not optional extras, they are the SDK.

► Functions & Return Values (Rank #1 for Spot)

- Write at least 5 reusable functions with parameters and return values.
- No floating code — everything lives inside a function called from a main block.

- Every SDK call — `robot.ensure_client()`, `command_client.robot_command()` — is a function call.

► If Statements & While Loops (Ranks #4 and #5)

- Write a while loop that polls a variable and breaks on a condition (simulate a sensor read loop).
- Write if/elif/else logic that responds to different states.
 - The estop check that keeps Spot safe is a while loop with a break condition.

► Classes & Objects (Rank #2 for Spot)

- Create a class that models a robot sensor. Give it at least 3 attributes and 2 methods.
- Instantiate two objects from the class with different attribute values.
- Call methods on those objects and print results.
 - The Robot class, LeaseClient, CommandClient — every Spot SDK object is exactly this pattern.

► Error Handling (Rank #3 for Spot)

- Wrap a risky operation (file read, network call simulation) in a try/except block.
- Catch at least two specific exception types — not bare except.
 - An unhandled exception on a moving robot is a safety incident.

Reference — Python video timestamps (Tier 1)

- **Functions (~1:35:00):** youtube.com/watch?v=rfscVS0vtbw&t=5700
- **Return statement (~1:50:00):** youtube.com/watch?v=rfscVS0vtbw&t=6600
- **While loops (~2:05:00):** youtube.com/watch?v=rfscVS0vtbw&t=7500
- **If statements (~1:55:00):** youtube.com/watch?v=rfscVS0vtbw&t=7020
- **Classes & objects (~3:05:00):** youtube.com/watch?v=rfscVS0vtbw&t=11100
- **Error handling (~2:45:00):** youtube.com/watch?v=rfscVS0vtbw&t=9900

Thursday — Day 4

Python — Dictionaries, File I/O & Reading Real Spot Data

 **Daily Standup 10:00 AM** — Standup as usual. Be specific about what you're building today.

Today you work with real robot data for the first time. You have been given `spot_patrol_data.csv` — 792 rows of telemetry from a Spot patrol mission across 11 waypoints. Your job today is to learn how to read it, understand it, and start pulling answers out of it. Friday's capstone uses this exact file.

YOUR DATA FILE

File: `spot_patrol_data.csv` — copy it into `/workspace/` inside your container.
 792 rows. 8 columns: timestamp, lap, waypoint, reading_num, sensor_id, value, unit, status.
 Status is either OK, FAULT_HIGH, or FAULT_LOW. Something went wrong on this mission. Your job today is to start finding it.

► Dictionaries — How Spot Data Is Structured (Rank #6)

- Read `spot_patrol_data.csv` using the `csv` module. Store each row as a dictionary in a list.
- Print the first 5 rows so you can see the dictionary structure clearly.
- Count how many unique `sensor_ids` exist in the file. Count how many unique waypoints.
 - `csv.DictReader` gives you each row as a dict automatically — this is the pattern you will use on the capstone.

► Modules & Imports (Rank #8)

- Import `csv`, `datetime`, and `collections` — all built-in, no pip needed.
- Use `collections.defaultdict` to group rows by `sensor_id`.
 - The Spot SDK is imported the same way. `import bosdyn.client` is line 1 of every real mission script.

► File I/O Practice — Find the Faults (Main Task)

- Write a function called `find_faults(rows)` that loops through your list of row dicts and returns only the rows where status is not OK.
- Call it and print how many faults you found and where they occurred.
- Write those fault rows to a new file called `day4_faults.csv` in `/workspace/week1/day4/`.
- Wrap all file operations in `try/except`.
 - There are 13 faults in the data. If you find a different number, check your loading logic.

► Per-Sensor Averages (Stretch Goal)

- Write a function called `sensor_averages(rows)` that calculates the average value for each `sensor_id`.
- Print results sorted highest to lowest, rounded to 2 decimal places.
 - This is Task 4 on the capstone. If you finish it today, Friday just got easier.

Reference — Python video timestamps (Tier 2)

- Dictionaries (~2:30:00): [youtube.com/watch?v=rfscVS0vtbw&t=9000](https://www.youtube.com/watch?v=rfscVS0vtbw&t=9000)
- Reading files (~2:55:00): [youtube.com/watch?v=rfscVS0vtbw&t=10500](https://www.youtube.com/watch?v=rfscVS0vtbw&t=10500)
- Writing & appending files (~3:00:00): [youtube.com/watch?v=rfscVS0vtbw&t=10800](https://www.youtube.com/watch?v=rfscVS0vtbw&t=10800)
- Modules (~3:15:00): [youtube.com/watch?v=rfscVS0vtbw&t=11700](https://www.youtube.com/watch?v=rfscVS0vtbw&t=11700)

Friday — Day 5

Capstone Day — Spot Patrol Data Analyzer

 **Daily Standup 10:00 AM** — Final standup. Tell us where you are and what you need to finish.

Today is capstone day. You have the project brief. You have the data file. Everything from this week comes together here — read the CSV, analyze it, find the faults, write the report, trigger it over SSH, push to GitHub.

TODAY'S GOAL

Build `analyzer.py` — five tasks, one script, one CSV file.

Tasks: load the data, mission summary, find all faults, per-sensor averages, write report file.

Trigger it over SSH. Push analyzer.py + mission_report.txt to GitHub by 5 PM.
Post your repo link in Teams. Demo Day is Monday June 29 at 10 AM.

► Start From Your Day 4 Work

- Your Day 4 code already loads the CSV and stores rows as dicts. Copy that function into analyzer.py — don't rewrite it.
- Open mission_report.txt from yesterday and remind yourself of the column structure before writing any logic.
 - Tasks 1 and 2 should take under 30 minutes if Day 4 went well. Get them done fast and move on.

► Build Tasks 1 Through 5 In Order

- Task 1: load CSV into a list of dicts, print row count. Working? Move on.
- Task 2: count unique laps, waypoints, sensor types from the data — not hardcoded.
- Task 3: find all rows where status != OK. Print each one. You should get 13 total.
- Task 4: average value per sensor_id, sorted highest to lowest, rounded to 2 decimal places.
- Task 5: write all of the above to mission_report.txt with clear section headers.
 - Every task is a function. Nothing floats at the top level.

► SSH Trigger — Do This Before You Push

- From your host machine (outside Docker), run:

```
ssh root@localhost -p 2222 'python3 /workspace/analyzer.py'
```

- If it works cleanly, you are done. If not, fix it now — not on Demo Day.
 - This is non-negotiable. If the SSH trigger does not work, the capstone is incomplete.

► GitHub — Push by 5 PM Friday June 27

- Create a repo: cast-capstone-[yourname]
- Push: analyzer.py and mission_report.txt
- README: what the script does, how to run it over SSH, what faults it found.
- Post your repo link in Teams by 5 PM Friday June 27.
 - Git for Beginners: [youtube.com/watch?v=RG0j5yH7evk](https://www.youtube.com/watch?v=RG0j5yH7evk)

End of Week 1 — Skill Benchmarks

These are the gates for Week 2 with Spot and the drones. If you can do all of these, you are ready.

Skill Area	By End of Week 1, You Can...
SSH & Remote Access	Configure an SSH server, generate keypairs, trigger remote Python execution without a password
Process Management	Locate and kill a runaway background process using PID — immediately

Linux Permissions	Read, set, and change file permissions using chmod and chown without looking it up
Python — Functions & Classes	Write modular functions with parameters/returns and create class instances with attributes and methods
Python — Control Flow	Write while loops with break conditions and if/elif/else logic that responds to state changes
Python — Error Handling	Wrap risky operations in try/except and catch specific exception types
Python — Data & Files	Read from and write to CSV files; structure telemetry data as dictionaries
Python — Modules	Import and use built-in Python modules (csv, datetime, random) in a working script
GitHub	Create a repo, commit daily, and push from the command line

Non-Negotiables

- You are expected to figure things out. Use the videos, use the docs, use your peers.
- Stuck for 20+ minutes? Follow the escalation path — don't sit silent.
- All work goes in GitHub. No repo = no record. No record = no accountability.
- 10:00 AM standup is not optional. Show up with something to say.
- Friday debrief is required. Communicate your progress like a professional.
- Come Monday ready to apply this — Week 2 is Spot SDK and drone SDK work.